

Extreme Value GAMs

GAM.jl Contributors

Table of contents

Introduction	1
Setup	1
GEV model	2
Simulate GEV data	2
Fit the GEV model	2
Examine parameter estimates	3
Compare fitted vs true	4
GPD model	5
Simulate GPD data	5
Fit the GPD model	6
Examine GPD estimates	6
Comparison with Julia	7

Introduction

This vignette demonstrates extreme value GAMs using the R **evgam** package, fitting the same simulated data as the Julia vignette for comparison.

Setup

```
library(evgam)
library(mgcv)
```

Loading required package: nlme

This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.

GEV model

Simulate GEV data

We generate the same data as the Julia vignette: block maxima with covariate-dependent location and scale.

```
set.seed(42)
n <- 500
x <- runif(n)

mu_true <- 5 + 2 * sin(2 * pi * x)
logsigma_true <- -0.5 + 0.5 * x
sigma_true <- exp(logsigma_true)
xi_true <- 0.1

rgev <- function(n, mu, sigma, xi) {
  u <- runif(n)
  if (abs(xi) < 1e-8) {
    return(mu - sigma * log(-log(u)))
  } else {
    return(mu + sigma * ((-log(u))^-xi - 1) / xi)
  }
}

y_gev <- sapply(1:n, function(i) rgev(1, mu_true[i], sigma_true[i], xi_true))

dat_gev <- data.frame(y = y_gev, x = x)
summary(dat_gev)
```

	y		x
Min.	: 1.864	Min.	:0.0002389
1st Qu.:	4.136	1st Qu.:	0.2320806
Median :	5.564	Median :	0.4803411
Mean :	5.537	Mean :	0.4895690
3rd Qu.:	6.816	3rd Qu.:	0.7433844
Max.	:13.312	Max.	:0.9965527

Fit the GEV model

In R `evgam`, we specify a list of formulas—one per distribution parameter.

```

m_gev <- evgam(
  list(y ~ s(x, k = 10, bs = "cr"),    # location (x)
        ~ s(x, k = 8, bs = "cr"),      # log-scale (x)
        ~ 1),                        # shape (constant)
  data = dat_gev,
  family = "gev"
)
summary(m_gev)

```

**** Parametric terms ****

location

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.03	0.04	132.1	<2e-16

logscale

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.29	0.04	-7.62	1.31e-14

shape

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.1	0.03	2.76	0.00287

**** Smooth terms ****

location

	edf	max.df	Chi.sq	Pr(> t)
s(x)	7.6	9	1718.1	<2e-16

logscale

	edf	max.df	Chi.sq	Pr(> t)
s(x)	1.07	7	3.87	0.0596

Examine parameter estimates

```

mu_hat <- predict(m_gev)$location
psi_hat <- predict(m_gev)$logscale
xi_hat <- coef(m_gev)

```

```
cat("Location fitted range:", range(mu_hat), "\n")
```

Location fitted range: 2.880489 7.035617

```
cat("Location true range:", range(mu_true), "\n")
```

Location true range: 3.000021 6.999916

```
cat("Correlation (location):", cor(mu_hat, mu_true), "\n")
```

Correlation (location): 0.9982123

```
cat("Log-scale fitted range:", range(psi_hat), "\n")
```

Log-scale fitted range: -0.4214744 -0.1656336

```
cat("Log-scale true range:", range(logsigma_true), "\n")
```

Log-scale true range: -0.4998806 -0.00172366

```
cat("Correlation (log-scale):", cor(psi_hat, logsigma_true), "\n")
```

Correlation (log-scale): 0.9995449

Compare fitted vs true

```
rmse_mu <- sqrt(mean((mu_hat - mu_true)^2))  
rmse_psi <- sqrt(mean((psi_hat - logsigma_true)^2))  
cat("RMSE (location):", round(rmse_mu, 3), "\n")
```

RMSE (location): 0.096

```
cat("RMSE (log-scale):", round(rmse_psi, 3), "\n")
```

RMSE (log-scale): 0.08

GPD model

Simulate GPD data

```
set.seed(42)
n_gpd <- 500
x_gpd <- runif(n_gpd)

logsigma_gpd_true <- 0.5 * sin(2 * pi * x_gpd)
sigma_gpd_true <- exp(logsigma_gpd_true)
xi_gpd_true <- 0.15

rgpd <- function(n, sigma, xi) {
  u <- runif(n)
  if (abs(xi) < 1e-8) {
    return(-sigma * log(u))
  } else {
    return(sigma * (u^(-xi) - 1) / xi)
  }
}

y_gpd <- sapply(1:n_gpd, function(i) rgpd(1, sigma_gpd_true[i], xi_gpd_true))

dat_gpd <- data.frame(y = y_gpd, x = x_gpd)
summary(dat_gpd)
```

	y		x
Min.	: 0.002488	Min.	:0.0002389
1st Qu.:	0.306916	1st Qu.:	0.2320806
Median	: 0.793762	Median	:0.4803411
Mean	: 1.319579	Mean	:0.4895690
3rd Qu.:	1.666570	3rd Qu.:	0.7433844
Max.	:15.840485	Max.	:0.9965527

Fit the GPD model

```
m_gpd <- evgam(  
  list(y ~ s(x, k = 10, bs = "cr"), # log-scale (x)  
        ~ 1), # shape (constant)  
  data = dat_gpd,  
  family = "gpd"  
)  
summary(m_gpd)
```

**** Parametric terms ****

```
logscale  
              Estimate Std. Error t value Pr(>|t|)  
(Intercept)    0.07      0.07      1.1    0.135
```

```
shape  
              Estimate Std. Error t value Pr(>|t|)  
(Intercept)    0.14      0.05      2.71 0.00341
```

**** Smooth terms ****

```
logscale  
      edf max.df Chi.sq Pr(>|t|)  
s(x) 4.12      9  37.94 2.04e-07
```

Examine GPD estimates

```
psi_gpd_hat <- predict(m_gpd)$logscale  
cat("Log-scale fitted range:", range(psi_gpd_hat), "\n")
```

Log-scale fitted range: -0.3945275 0.4570417

```
cat("Log-scale true range:", range(logsigma_gpd_true), "\n")
```

Log-scale true range: -0.4999947 0.4999791

```
cat("Correlation (log-scale):", cor(psi_gpd_hat, logsigma_gpd_true), "\n")
```

Correlation (log-scale): 0.9569373

Comparison with Julia

The R `evgam` and Julia GAM.jl `evgam` functions share the same interface structure:

R <code>evgam</code>	Julia GAM.jl
<code>evgam(list(y ~ s(x), ~ s(x), ~ 1), family="gev")</code>	<code>evgam([@gam_formula(y ~ s(x)), @gam_formula(y ~ s(x)), @gam_formula(y ~ 1)], GEVFamily())</code>
<code>predict(m)\$location</code>	<code>param_eta(m, 1)</code>
<code>predict(m)\$logscale</code>	<code>param_eta(m, 2)</code>
<code>family = "gev"</code>	<code>GEVFamily()</code>
<code>family = "gpd"</code>	<code>GPDFamily()</code>